

INVENTORY TRACKER

Project Specification Document

Version	1.0
Status	Draft
Author	FutureSmart AI
Last Updated	June 2026
Confidentiality	Internal

1. Document Information & Executive Summary

Document Information

Field	Value
Project Name	Inventory Tracker
Document Version	1.0
Status	Draft
Author(s)	FutureSmart AI Engineering Team
Reviewer(s)	Product Manager, Engineering Manager
Approver(s)	CTO
Last Updated	June 28, 2026
Confidentiality	Internal — Not for External Distribution
Change Log	v1.0 — Initial draft covering all core specification sections

Executive Summary

The Inventory Tracker is a full-stack web application for small and medium businesses to manage product stock, track movements, and generate reports — replacing fragmented spreadsheets with a real-time, role-aware system.

Problem: Businesses using spreadsheets face stockouts, duplicate entries, zero real-time visibility, and hours of weekly reconciliation work.

Solution: A React + FastAPI + Supabase web app with real-time stock tracking, automated low-stock alerts, multi-user RBAC, transaction history, and exportable reports.

Goal Type	Goal
Business	Reduce stockout incidents by 80% within 6 months of launch
Business	Save each team 5+ hours/week previously spent on manual reconciliation
Technical	Sub-200ms API response (p95) for all read operations
Technical	Zero-downtime deployments via blue-green strategy
UX	New users onboard and complete first stock entry in under 10 minutes

2. Background & Vision

Existing System: Most targets use Excel/Google Sheets or legacy desktop apps (Tally) with no web or multi-user access.

Pain Points:

- No real-time stock visibility — data is hours or days stale
- Concurrent edits in shared spreadsheets cause data corruption and lost changes
- No audit trail — impossible to trace who changed what and when
- Low-stock alerts require constant manual checking
- Reporting is time-consuming pivot-table work done monthly at best

Vision: By Q4 2026, every business user — from a solo shop owner to a multi-warehouse operations team — can open a browser, see their exact stock in real time, act on automated alerts, and close monthly reports in minutes rather than days.

3. Objectives & Scope

Objectives

Category	Objective	Success Metric
Business	Eliminate manual stock reconciliation	Reconciliation time < 5 min/day
Business	Provide real-time low-stock alerts	Alert latency < 60 seconds after breach
Technical	RESTful API with full CRUD on all entities	100% endpoint coverage with tests
Technical	JWT-based auth with RBAC	0 unauthorized access incidents in pen-test
Performance	Fast page loads	LCP < 2.5s, TTFB < 200ms
Security	Data protection	HTTPS enforced, encrypted secrets, audit logs
UX	Intuitive interface for non-technical users	SUS score >= 80 in usability testing

Scope

In Scope	Out of Scope
Product & category management	Native mobile app (iOS/Android)
Inventory stock tracking with transactions	POS hardware integration
Multi-location warehouse support	Accounting/ERP integration (v2 roadmap)
Low-stock alerts & email notifications	Multi-currency support (v2)
Role-based access: Admin, Manager, Staff, Viewer	AI demand forecasting (v2)
Transaction history & full audit log	Customer-facing order portal
CSV/Excel and PDF export	Offline-first PWA
Supplier management (basic directory)	Barcode hardware scanner SDK
Dashboard with KPI widgets	

4. Stakeholders & User Personas

Stakeholders

Role	Responsibility
Product Manager	Requirements, prioritisation, sign-off
Engineering Manager	Architecture decisions, code review, delivery
Frontend Team (x2)	React UI, component library, accessibility
Backend Team (x2)	FastAPI services, DB design, migrations
QA Engineer	Test plans, automation, regression, load testing
DevOps Engineer	CI/CD, infrastructure, monitoring, secrets
End Users	Day-to-day inventory operations (warehouse staff, managers)
Security Reviewer	Pre-launch pen-test and compliance review

User Personas

Arjun Sharma — Warehouse Manager

Age / Role	34 · Manages 2 warehouses and 8 staff
Goals	Real-time stock visibility, low-stock alerts, quick management reports
Pain Points	Discovers stockouts after complaints; chases staff for manual updates; no audit trail
Tech Proficiency	Comfortable with web apps — desktop + tablet
Primary Journey	Login → Dashboard → Review alerts → Generate report → Export PDF

Priya Nair — Store Clerk

Age / Role	24 · Records daily stock-in and stock-out transactions
Goals	Fast, error-free stock entry with minimal clicks and clear confirmation
Pain Points	Slow spreadsheets, duplicate entries, no feedback when entry is saved
Tech Proficiency	Basic — uses shared desktop and phone
Primary Journey	Login → Products → Add Transaction → Confirm → Done

Meera Kapoor — Business Owner

Age / Role	47 · Reviews KPIs monthly, approves purchase orders
Goals	High-level inventory health view and cost insights without digging into data
Pain Points	No single source of truth; inconsistent reports from different staff
Tech Proficiency	Low — prefers simple dashboards, uses iPad
Primary Journey	Login → Dashboard → Click KPI chart → Download report

5. Functional Requirements

Each requirement has a unique ID, priority (High/Medium/Low), and measurable acceptance criteria.

ID	Title	Description	Pri	Acceptance Criteria
FR-01	User Authentication	Email + password login with JWT; session expires 30 mins. Invalid credentials return 401. Expired tokens return 403.	High	Valid credentials issue token. Invalid credentials return 401. Expired tokens return 403.
FR-02	User Invitation	Admin invites users by email. Invited user sets password first login. First-login flow works. Duplicate invite rejected.	Medium	Invite email sent with link. User sets password on first login. Duplicate invite rejected with 400.
FR-03	RBAC	Four roles: Admin, Manager, Staff, Viewer. Permissions enforced at server level. Role changes take effect on next login.	High	Permissions enforced at server level. Role changes take effect on next login.
FR-04	Product Management	CRUD on products with name, SKU, category, unit price. Product deleted/soft-deleted. Duplicate SKU rejected. History preserved.	High	Product created, updated, deleted. Duplicate SKU rejected. History preserved.
FR-05	Category Management	Create and manage product categories with parent/child hierarchy. Products assignable to any category. Category deletion cascades.	Medium	Categories created. Products assignable to any category. Category deletion cascades.
FR-06	Inventory Stock Levels	Track current quantity per product per warehouse. Stock updates atomically on each transaction. Negative stock blocked.	High	Stock updates atomically on each transaction. Negative stock blocked.
FR-07	Stock Transactions	Record stock-in, stock-out, transfer with quantity and notes. Types work. Quantity reflects immediately. Each transaction time-stamped.	High	Transactions recorded with quantity and notes. Quantity reflects immediately. Each transaction time-stamped.
FR-08	Low-Stock Alerts	Trigger in-app + email alerts when stock falls below configured level. Alerts fire within 60s of threshold breach. Email delivered within 2 min.	Medium	Alerts fire within 60s of threshold breach. Email delivered within 2 min.
FR-09	Supplier Management	Supplier directory with name, contact, email, products. Suppliers associated and linked to products. Supplier list searchable by name.	Medium	Suppliers associated and linked to products. Supplier list searchable by name.
FR-10	Purchase Orders	Create, track, and receive POs against suppliers. PO created with line items. Receiving PO auto-updates stock levels. Status tracking.	High	PO created with line items. Receiving PO auto-updates stock levels. Status tracking.
FR-11	Multi-Location Warehouse	Separate stock levels per warehouse. Transfers between locations. Stock levels tracked independently per location. Transfer debits/credits auto-calculated.	High	Stock levels tracked independently per location. Transfer debits/credits auto-calculated.
FR-12	Dashboard	KPI widgets: total products, low-stock count, daily movements, current stock data. Charts render correctly. Data current to last 15 min.	Medium	Charts render correctly. Data current to last 15 min.
FR-13	Transaction History	Paginated log of all movements with filters by date, product, user. 50/page. Date, product, and user filters work independently.	Medium	Log of all movements with filters by date, product, user. 50/page. Date, product, and user filters work independently.
FR-14	CSV / Excel Export	Export inventory list and transaction history. CSV/Excel export in Excel without errors. Includes all displayed columns. Responsive.	Medium	CSV/Excel export in Excel without errors. Includes all displayed columns. Responsive.
FR-15	PDF Reports	Server-generated inventory summary and transaction report. Includes header, date range, totals. Downloadable within 5s. Correct formatting.	Medium	PDF reports generated. Includes header, date range, totals. Downloadable within 5s. Correct formatting.
FR-16	Audit Log	Immutable record of every data-mutating action (create, update, delete). Entry includes user, IP, timestamp. Admin-only access.	High	Entry created for every CREATE/UPDATE/DELETE. Admin-only access.
FR-17	Search & Filter	Global product search. Filter by category, location, stock status. Results within 300ms. Filters are combinable. Partial match support.	Medium	Search results within 300ms. Filters are combinable. Partial match support.
FR-18	Notifications Centre	In-app panel listing unread alerts and system events. Bell icon shows unread count. Mark-all-read works. Notifications persist across sessions.	Medium	Bell icon shows unread count. Mark-all-read works. Notifications persist across sessions.

6. Non-Functional Requirements

Category	Requirement	Target
Performance	API read response time (p95)	< 200ms
Performance	API write response time (p95)	< 500ms
Performance	Largest Contentful Paint (LCP)	< 2.5s
Performance	Time to First Byte (TTFB)	< 200ms
Availability	System uptime (monthly)	≥ 99.5%
Reliability	Successful request rate	≥ 99.9%
Scalability	Concurrent users without degradation	500 concurrent
Scalability	Data volume per tenant	Up to 500,000 products
Security	Data in transit	TLS 1.2+ (HTTPS enforced via HSTS)
Security	Data at rest	AES-256 via Supabase encryption
Security	Secrets management	Environment variables only; no hardcoded keys
Maintainability	Test coverage	≥ 80% BE unit+integration; ≥ 75% FE
Accessibility	WCAG compliance	Level 2.1 AA on all pages
Compatibility	Browser support	Chrome 110+, Firefox 112+, Safari 15+, Edge 110+
Compatibility	Responsive design breakpoints	1280px desktop, 768px tablet, 375px mobile
Localization	Initial locale	English (en-IN); dates and currency formatted for India
Monitoring	Error tracking SLA	Sentry; P1 alert within 5 min of occurrence
Backup	Database backup frequency	Daily automated; 30-day retention
DR	Recovery Time Objective (RTO)	< 4 hours for full outage
DR	Recovery Point Objective (RPO)	< 24 hours (daily backup cadence)

7. System Architecture & Technology Stack

Architecture Overview

Three-tier architecture: React SPA (frontend) → FastAPI REST API (backend) → Supabase PostgreSQL (database).
Real-time low-stock alerts use Supabase Realtime (PostgreSQL LISTEN/NOTIFY over WebSocket). Auth uses JWT access tokens (15-min TTL) with refresh tokens in HttpOnly cookies.

Layer	Component	Responsibility
Client	React 18 SPA (Vite)	UI rendering, state, API calls, real-time WebSocket
API	FastAPI + Uvicorn	REST endpoints, request validation, auth middleware
Business Logic	FastAPI services	CRUD, alert engine, report generation, background tasks
Database	Supabase PostgreSQL 15	Persistent storage, RLS policies, triggers, PITR
Realtime	Supabase Realtime	Low-stock alerts broadcast via WebSocket
Storage	Supabase Storage	Product images, generated PDF reports
Auth	Custom JWT + Supabase Auth	Login, token issuance, RBAC enforcement
Email	Resend (SMTP fallback)	Transactional: alerts, invitations, password reset
CDN/Hosting (FE)	Vercel	Global edge delivery of static assets
Hosting (BE)	Railway / Render	Containerised FastAPI (Docker) deployment
Cache	Upstash Redis	Dashboard stats TTL 5 min; report data TTL 15 min
Monitoring	Sentry + Grafana Cloud	Error tracking, performance metrics, dashboards
Analytics	PostHog (self-hosted option)	Event tracking, funnels, A/B testing

Technology Stack

Layer	Technology	Version / Notes
Frontend Framework	React	18.x with Hooks
Language (FE)	TypeScript	5.x — strict mode
Build Tool	Vite	5.x
Styling	Tailwind CSS + shadcn/ui	3.x component library
Server State	TanStack Query (React Query)	5.x — caching, refetch, optimistic updates
Global State	Zustand	4.x — auth, UI, alerts
HTTP Client	Axios	1.x — interceptor for token refresh
Forms & Validation	React Hook Form + Zod	Schema-driven validation
Charts	Recharts	2.x
Backend Framework	FastAPI	0.111.x
Language (BE)	Python	3.12
ORM	SQLAlchemy 2.0	Async + asyncpg driver

Layer	Technology	Version / Notes
Database	PostgreSQL 15	Via Supabase managed service
Auth (BE)	python-jose	RS256 JWT signing
PDF Generation	ReportLab	Server-side; async background task
CI/CD	GitHub Actions	Lint → test → build → deploy pipeline
Testing (FE)	Vitest + React Testing Library	Unit and integration
Testing (BE)	Pytest + httpx	Unit, integration, async
E2E Testing	Playwright	Critical user flow coverage
Security Scanning	OWASP ZAP (DAST)	Run against staging before each release
Containerisation	Docker	Multi-stage Dockerfile; slim Python image

8. Folder Structure

Frontend (React/Vite)

```
inventory-tracker/
frontend/
  src/
    api/           # Axios instances + API call functions per domain
    assets/        # Static images, icons, fonts
    components/    # Shared reusable UI (Button, Modal, Table, Badge...)
    features/      # Feature-scoped modules
      products/    # ProductList, ProductForm, useProducts (React Query hooks)
      inventory/   # StockLevels, TransactionForm, useInventory
      suppliers/   # SupplierList, SupplierForm
      reports/     # ReportBuilder, ExportPanel
      dashboard/   # KPIWidgets, AlertStrip, StockChart
    hooks/         # Global custom hooks (useAuth, useAlert, useDebounce)
    layouts/       # Page shells: AppLayout, AuthLayout
    pages/         # Route-level thin wrappers around feature components
    store/         # Zustand slices: authStore, alertStore, uiStore
    types/         # Global TypeScript interfaces and enums
    utils/         # Pure helpers: formatDate, formatCurrency, paginate
    config/        # Constants, route paths, env vars
    App.tsx        # Root component with React Router
    main.tsx       # Entry point
  public/
  tests/
    unit/          # Vitest unit tests
    e2e/           # Playwright E2E tests
  vite.config.ts
  tailwind.config.ts
  tsconfig.json
```

Backend (FastAPI)

```
backend/
  app/
    api/
      v1/
        routes/      # One file per domain: auth.py, products.py, inventory.py...
        dependencies.py # FastAPI DI: get_db, get_current_user, require_role
    core/
      config.py       # pydantic-settings: all env vars typed and validated
      security.py      # JWT creation, verification, password hashing
      database.py      # Async SQLAlchemy engine + session factory
    models/           # SQLAlchemy ORM models (products.py, users.py...)
    schemas/          # Pydantic v2 request/response schemas
    services/         # Business logic: inventory_service.py, alert_service.py...
    tasks/            # Background tasks: alert_engine.py, report_gen.py
    utils/            # Shared: pagination.py, pdf_generator.py
    main.py           # FastAPI app factory, middleware, router registration
  migrations/         # Alembic migration files
  tests/
    unit/
    integration/
  Dockerfile
  requirements.txt
  alembic.ini
```

9. Database Design & Data Models

PostgreSQL 15 via Supabase. Row-Level Security (RLS) policies provide second-layer authorisation. All tables have created_at / updated_at. Soft-delete via is_active flag on products and suppliers preserves referential integrity in transaction history.

Tables Overview

Table	Description	Key Relationships
users	Application users with role, status, org	Has many transactions, audit_logs
warehouses	Physical storage locations	Has many inventory_levels
categories	Product category tree (self-referencing parent_id)	Has many products
suppliers	Supplier directory	Has many products, purchase_orders
products	Product catalogue with SKU, reorder level	FK to category, supplier; has many inventory_levels
inventory_levels	Current stock qty per product per warehouse	FK to product, warehouse; UNIQUE(product, warehouse)
transactions	All stock movements — immutable append-only	FK to product, warehouse, user
purchase_orders	PO header with status lifecycle	FK to supplier; has many po_items
po_items	Line items on a purchase order	FK to purchase_order, product
audit_logs	Immutable record of all data mutations	FK to user; non-deletable
notifications	System-generated alerts per user	FK to user, product

Data Models

Table: products

Column	Type	Notes
id	UUID	PK, gen_random_uuid()
sku	VARCHAR(50)	UNIQUE NOT NULL
name	VARCHAR(200)	NOT NULL
description	TEXT	Nullable
category_id	UUID	FK → categories.id
supplier_id	UUID	FK → suppliers.id, Nullable
unit	VARCHAR(20)	e.g. 'pcs', 'kg', 'litre'
unit_cost	DECIMAL(12,2)	Default 0.00
reorder_level	INT	Triggers low-stock alert when stock falls below
image_url	TEXT	Supabase Storage URL, Nullable
is_active	BOOLEAN	Soft-delete flag; default TRUE
created_at	TIMESTAMPTZ	Default NOW()
updated_at	TIMESTAMPTZ	Auto-updated via trigger

Table: inventory_levels

Column	Type	Notes
id	UUID	PK
product_id	UUID	FK → products.id NOT NULL
warehouse_id	UUID	FK → warehouses.id NOT NULL
quantity	DECIMAL(12,2)	Default 0.00; CHECK >= 0 (configurable)
UNIQUE	(product_id, warehouse_id)	Composite unique constraint
updated_at	TIMESTAMPTZ	Auto-updated on each transaction

Table: transactions

Column	Type	Notes
id	UUID	PK
type	ENUM('stock_in','stock_out','transfer')	NOT NULL
product_id	UUID	FK → products.id NOT NULL
from_warehouse_id	UUID	FK → warehouses; NULL for stock_in
to_warehouse_id	UUID	FK → warehouses; NULL for stock_out
quantity	DECIMAL(12,2)	NOT NULL, CHECK > 0
notes	TEXT	Nullable — operator notes
reference_no	VARCHAR(100)	PO/delivery reference; Nullable
created_by	UUID	FK → users.id NOT NULL
created_at	TIMESTAMPTZ	Default NOW() — immutable once written

Table: purchase_orders

Column	Type	Notes
id	UUID	PK
supplier_id	UUID	FK → suppliers.id NOT NULL
status	ENUM('draft','submitted','partial','received','cancelled')	Default: draft
expected_date	DATE	Nullable
notes	TEXT	Nullable
created_by	UUID	FK → users.id
created_at	TIMESTAMPTZ	Default NOW()
updated_at	TIMESTAMPTZ	Auto-updated

Indexes

Table	Index Name	Columns	Type
products	idx_products_sku	sku	UNIQUE BTREE
products	idx_products_category	category_id	BTREE

Table	Index Name	Columns	Type
products	idx_products_supplier	supplier_id	BTREE
transactions	idx_txn_product_date	product_id, created_at DESC	BTREE
transactions	idx_txn_warehouse	from_warehouse_id, to_warehouse_id	BTREE
inventory_levels	idx_inv_product_wh	product_id, warehouse_id	UNIQUE
audit_logs	idx_audit_entity	entity_type, entity_id	BTREE
audit_logs	idx_audit_user_date	user_id, created_at DESC	BTREE

10. API Specification

Base URL: <https://api.inventorytracker.app/api/v1> | Auth: **Authorization: Bearer {token}** unless marked Public. | All responses: JSON. | Errors: RFC 7807 Problem Details format. | Rate limit: 300 req/min authenticated; 20 req/min on /auth/login.

Method	Endpoint	Auth	Description	Success
POST	/auth/login	Public	Email+password login	access_token, refresh_token, user
POST	/auth/refresh	Cookie	Rotate refresh token	New access_token
POST	/auth/logout	Bearer	Revoke refresh token	{ message: "ok" }
POST	/auth/forgot-password	Public	Send reset email	204 No Content
POST	/auth/reset-password	Public	Set new password with reset token	200 OK
POST	/auth/invite	Admin	Invite user by email	{ invitation_id }
GET	/users	Admin	List all users (paginated)	Paginated user list
GET	/users/me	Bearer	Get current user profile	User object
PATCH	/users/{id}/role	Admin	Change user role	Updated user
PATCH	/users/{id}/status	Admin	Activate/deactivate user	Updated user
GET	/products	Bearer	List products (paginated, filterable)	Paginated products
POST	/products	Manager+	Create product	Created product (201)
GET	/products/{id}	Bearer	Product detail + current stock levels	Product + inventory
PATCH	/products/{id}	Manager+	Update product fields	Updated product
DELETE	/products/{id}	Admin	Soft-delete product	204 No Content
GET	/categories	Bearer	List category tree	Category tree array
POST	/categories	Manager+	Create category	Created category (201)
DELETE	/categories/{id}	Admin	Delete if no products assigned	204 No Content
GET	/inventory	Bearer	Stock levels, filter by product/warehouse	Inventory list
POST	/transactions	Staff+	Record stock movement	Created transaction (201)
GET	/transactions	Bearer	History — paginated, filterable	Transactions + metadata
GET	/warehouses	Bearer	List warehouses	Warehouse list
POST	/warehouses	Admin	Create warehouse	Created warehouse (201)
PATCH	/warehouses/{id}	Admin	Update warehouse details	Updated warehouse
GET	/suppliers	Bearer	List suppliers	Supplier list
POST	/suppliers	Manager+	Create supplier	Created supplier (201)
PATCH	/suppliers/{id}	Manager+	Update supplier	Updated supplier
GET	/purchase-orders	Bearer	List POs with status	Paginated PO list
POST	/purchase-orders	Manager+	Create PO with line items	Created PO (201)

Method	Endpoint	Auth	Description	Success
PATCH	/purchase-orders/{id}/receive	Staff+	Mark received; auto-update stock	Updated PO
PATCH	/purchase-orders/{id}/cancel	Manager+	Cancel PO	Updated PO
GET	/reports/inventory	Bearer	Inventory snapshot data	Report payload
GET	/reports/inventory/pdf	Bearer	Download inventory PDF	PDF binary stream
GET	/reports/transactions/pdf	Bearer	Download transactions PDF	PDF binary stream
GET	/dashboard/stats	Bearer	KPI summary object	Stats + alerts
GET	/audit-logs	Admin	Paginated audit trail	Audit entries
GET	/notifications	Bearer	User notifications	Notification list
PATCH	/notifications/read-all	Bearer	Mark all read	{ count: N }

Standard Error Body: { error: string, detail: string | object[], status: number, timestamp: ISO8601 }

11. Authentication & Authorization

Auth Flow

- POST /auth/login → server validates credentials + is_active flag
- Issues: access_token (JWT RS256, 15-min TTL) + refresh_token (opaque UUID, 7-day TTL, HttpOnly SameSite=Strict cookie)
- Client stores access_token in memory only (not localStorage — XSS protection)
- Axios interceptor attaches Bearer token to every request header
- On 401: interceptor calls POST /auth/refresh before retrying original request transparently
- On logout or tab close: POST /auth/logout revokes refresh token server-side
- Supabase RLS provides second enforcement layer at DB level (defence in depth)

Password Policy

- Minimum 8 characters; must include uppercase, lowercase, digit, special character
- Bcrypt hashing with cost factor 12
- Failed login rate limit: 5 attempts per IP per 15 minutes → temporary lockout
- Forgot password: time-limited token (15 min) sent to verified email only
- MFA (TOTP via Google Authenticator): optional for all roles; mandatory for Admin in v1.1

Authorization Matrix

Feature / Action	Admin	Manager	Staff	Viewer
Login / Logout	Y	Y	Y	Y
View Dashboard & KPIs	Y	Y	Y	Y
View Products & Stock	Y	Y	Y	Y
Create / Edit Products	Y	Y	N	N
Delete Products (soft)	Y	N	N	N
Record Stock-In / Stock-Out	Y	Y	Y	N
Record Stock Transfers	Y	Y	Y	N
Manage Warehouses	Y	N	N	N
Manage Categories	Y	Y	N	N
Manage Suppliers	Y	Y	N	N
Create Purchase Orders	Y	Y	N	N
Receive Purchase Orders	Y	Y	Y	N
View Reports & Export	Y	Y	Y	Y
Generate PDF Reports	Y	Y	N	N
Invite / Manage Users	Y	N	N	N
Change User Roles	Y	N	N	N
View Audit Logs	Y	N	N	N

Feature / Action	Admin	Manager	Staff	Viewer
System Settings	Y	N	N	N

12. UI / UX Requirements

Pages & Routes

Route	Layout	Description	Auth
/login	AuthLayout	Email + password form, forgot-password link	No
/dashboard	AppLayout	KPI cards, charts, low-stock alert strip	Yes
/products	AppLayout	Paginated product table with search and filters	Yes
/products/new	AppLayout	Multi-step product creation form	Manager+
/products/:id	AppLayout	Product detail, stock history, edit option	Yes
/inventory	AppLayout	Stock levels per product per warehouse	Yes
/transactions	AppLayout	Transaction log with date/product/user filters	Yes
/transactions/new	AppLayout	Stock movement form (in / out / transfer)	Staff+
/warehouses	AppLayout	Warehouse management	Admin
/suppliers	AppLayout	Supplier directory with linked products	Yes
/purchase-orders	AppLayout	PO list and status tracker	Yes
/purchase-orders/new	AppLayout	Create PO with line items	Manager+
/reports	AppLayout	Report builder: date range, filters, export	Yes
/settings/users	AppLayout	User management and role assignment	Admin
/settings/account	AppLayout	Profile, password change, notification prefs	Yes

UX Principles

- Progressive disclosure — only show controls relevant to the current user's role
- Optimistic updates — record transaction in UI immediately; roll back on server error with toast
- Persistent filter state — active filters stored in URL query params; survive page refresh
- Empty states with clear CTAs — no blank table without guidance text and an action button
- Confirmation modals for all destructive actions (delete product, deactivate user, cancel PO)
- Toast notifications for all async operations (success, error, loading indicators)
- Skeleton loaders instead of spinners for table-heavy and data-heavy pages
- Keyboard navigation for all interactive elements (WCAG 2.1 AA compliance)
- Responsive from 375px mobile to 1440px+ desktop; sidebar collapses to hamburger below 768px

13. Design System

Color Palette

Token	Hex	Usage
color-primary	#1A56DB	Primary actions, links, active states, focus rings
color-primary-light	#EFF6FF	Tinted backgrounds, selected rows, info banners
color-navy	#0C1A2E	Primary text, headings, nav background
color-slate	#475569	Secondary text, captions, placeholder text
color-border	#E2E8F0	Dividers, table borders, input borders
color-surface	#F5F7FA	Page background, alternating table rows
color-success	#10B981	In-stock, positive deltas, resolved states
color-warning	#F59E0B	Low-stock alerts, medium-priority badges, caution
color-danger	#EF4444	Out-of-stock, form errors, critical alerts, delete
color-white	#FFFFFF	Card backgrounds, modal backgrounds, input fill

Typography Scale

Token	Size / Weight	Usage
text-display	28px / 800	Page hero titles, large empty state headings
text-h1	22px / 700	Primary page heading (one per page)
text-h2	16px / 700	Section headings, card titles, modal titles
text-h3	14px / 600	Subsection headings, table group labels
text-body	14px / 400	Default body text, form labels, table cell content
text-small	12px / 400	Captions, timestamps, secondary metadata
text-mono	13px / 400 Courier	SKUs, IDs, code values, quantities
text-badge	11px / 700 uppercase	Status badges, category chips

Component Library

Component	Variants	Notes
Button	Primary, Secondary, Ghost, Danger, Icon-only	Loading spinner state; disabled state
Input	Text, Number, Password, Textarea, Search	Error state, helper text, prefix/suffix icon
Select	Single, Multi, Searchable	Built on react-select for complex cases
Table	Default, Condensed, Sortable	Pagination, row selection, sticky header
Badge / Pill	Success, Warning, Danger, Info, Neutral	Used for status labels and category chips
Card	Default (border), Flat, Interactive (hover)	8px radius; 1px border; subtle shadow
Modal	SM 400px, MD 600px, LG 800px	Focus trap; ESC to close; backdrop click closes

Component	Variants	Notes
Toast	Success, Error, Warning, Info	Auto-dismiss 4s; manual dismiss X button
Skeleton	Text line, Block, Table row	Pulse animation; matches shape of real content
Alert	Inline, Banner	Dismissable; icon + message + optional action link
DatePicker	Single date, Date range	Locale-aware (en-IN); keyboard accessible
Empty State	Icon + heading + body + CTA	Consistent empty state pattern across all pages

Spacing scale (base 4px): 4, 8, 12, 16, 20, 24, 32, 40, 48, 64, 80px. **Border radius:** 4px (sm), 8px (md/default), 12px (lg), 9999px (pill). **Elevation:** 3 levels — card (0 2px 8px rgba(0,0,0,0.06)), dropdown (0 4px 16px rgba(0,0,0,0.12)), modal (0 8px 40px rgba(0,0,0,0.18)).

14. State Management & Validation

State Strategy

State Type	Tool	Holds
Server / Remote State	TanStack Query v5	All API data. Caching, background refetch, optimistic updates.
Global UI State	Zustand	Auth user + token, notification count, sidebar state, active warehouse.
Local Component State	React useState / useReducer	Form fields, modal open, active tab, page number.
URL / Navigation State	React Router URLSearchParams	Active filters, search, pagination — survives refresh, enables shared URLs.
Form State	React Hook Form	Field values, dirty/touched, errors, submission lifecycle.
Persistence	localStorage (selective)	User preferences (theme, preferred warehouse). Auth token: memory only.

Validation Rules

Field	Rule	Error Message
Email	RFC 5322 regex; max 254 chars	"Enter a valid email address."
Password (new)	Min 8 chars; 1 upper, 1 lower, 1 digit, 1 special char	"Must be 8 chars with uppercase, lowercase, number, and symbol."
Product Name	Non-empty; max 200 chars	"Product name is required (max 200 characters)."
SKU	Non-empty; max 50; alphanumeric + dash/underscore	"SKU must be unique — letters, numbers, dashes, underscores only."
Quantity (transaction)	Positive decimal; max 9,999,999; 2dp	"Enter a positive quantity."
Reorder Level	Non-negative integer	"Reorder level must be 0 or greater."
Unit Cost	Non-negative decimal; max 2dp	"Enter a valid positive amount."
Category	Must select from existing list	"Select a valid category."
Warehouse (transaction)	Required for all transaction types	"Select a warehouse."
Transfer: from/to	from_warehouse_id != to_warehouse_id	"Source and destination warehouse must differ."
PO Line Quantity	Positive integer	"Enter a valid quantity greater than zero."
Supplier Email	Valid email format if provided	"Enter a valid email address."

15. Error Handling & Security

Error Handling Strategy

Scenario	Client Behaviour	User-Facing Message
Network timeout / offline	Retry 3x with exponential backoff; show offline banner	"Connection lost. Retrying..."
401 Unauthorized	Attempt token refresh; redirect /login if refresh fails	"Session expired. Please log in again."
403 Forbidden	Block action; no redirect	"You don't have permission for this action."
404 Not Found	Render 404 page component	"The resource was not found."
422 Validation Error	Map server field errors directly to form fields	Per-field error messages under inputs
429 Rate Limited	Disable submit for retry-after duration; show warning toast	"Too many requests. Please wait before trying again."
500 Server Error	Show generic error toast; log to Sentry	"Something went wrong. Our team has been notified."
Optimistic update fail	Revert UI to previous state; show error toast	"Failed to save. Your change was undone."

Security Requirements

- HTTPS enforced on all environments; HSTS header with max-age=31536000 (1 year)
- JWT signed with RS256 (asymmetric keys) — private key stored in secrets manager, never in code
- Access tokens: memory only (never localStorage); refresh tokens: HttpOnly, SameSite=Strict, Secure cookie
- CSRF: SameSite=Strict cookie + custom X-Requested-With header check on state-mutating endpoints
- XSS: React JSX escaping by default; Content-Security-Policy header blocks inline scripts
- SQL injection: all queries use parameterized statements via SQLAlchemy ORM
- Rate limiting: 20 req/min on /auth/login; 5 failed attempts → 15-min logout per IP
- Input sanitization: Pydantic v2 strips unexpected fields on all API inputs
- Supabase RLS policies enforce tenant data isolation at DB level (defence in depth)
- Audit log for every CREATE/UPDATE/DELETE — immutable, append-only, admin-read-only
- Dependency scanning: Dependabot weekly PRs; security patches merged within 48h
- Security headers via middleware: X-Frame-Options: DENY, X-Content-Type-Options: nosniff, Referrer-Policy: no-referrer
- Secret management: all secrets via environment variables; Doppler or Railway secrets UI for production

16. Performance & Scalability

Core Web Vitals Targets

Metric	Target	Tool
Largest Contentful Paint (LCP)	< 2.5s	Lighthouse, CrUX
Interaction to Next Paint (INP)	< 200ms	Web Vitals extension, Sentry
Cumulative Layout Shift (CLS)	< 0.1	Lighthouse
Time to First Byte (TTFB)	< 200ms	Server logs, Cloudflare analytics
API reads (p50)	< 100ms	FastAPI instrumentation
API reads (p95)	< 200ms	Sentry Performance, Grafana
API writes (p95)	< 500ms	Sentry Performance, Grafana
JS bundle (initial, gzipped)	< 200KB	Vite bundle analyser, rollup-visualizer
Product images	WebP format, max 150KB each	Supabase Image Transform on upload

Scalability Plan

- Horizontal scaling: stateless FastAPI containers behind a load balancer (Railway or AWS ECS); scale on CPU > 70%
- DB connection pooling: PgBouncer in transaction mode; max 100 connections per service instance to avoid Supabase limit
- Query optimisation: EXPLAIN ANALYSE run on all list endpoints before production; slow query log enabled
- Caching layer: Upstash Redis for dashboard stats (TTL 5 min) and report pre-computation (TTL 15 min)
- CDN: Vercel Edge Network for all static assets; API responses with Cache-Control: stale-while-revalidate where safe
- Background jobs: PDF generation and bulk exports offloaded to FastAPI BackgroundTasks (v1); migrate to Celery + Redis in v2
- Read replicas: Supabase read replica enabled for all reporting and export queries from v1.1
- Code splitting: React lazy() + Suspense on all route-level components; feature chunks loaded on demand

17. Testing Strategy

Type	Tool	Coverage Target	Scope
Unit — Backend	Pytest	≥ 80% line coverage	Pure functions, service layer, validation. Mocked DB via test fixtures.
Unit — Frontend	Vitest	≥ 75% line coverage	Utility functions, Zustand stores, custom hooks. No DOM rendering.
Integration — Backend	Pytest + httpx + test DB	All API endpoints	Full request-response against a real test database. Transactions rolled back after tests.
Integration — Frontend	React Testing Library + MSW	All major components	Component render, user interaction, API mocked via MSW service worker.
End-to-End	Playwright	Critical user flows	Login, add product, record transaction, generate report, logout. Run on every PR.
Load / Performance	k6	Key read endpoints	500 virtual users for 5 min on /products, /inventory, /transactions. Pass: p95 < 2s.
Security (DAST)	OWASP ZAP	Full app surface on staging	Automated scan before each release. Block release on any High/Critical finding.
Accessibility	axe-core + Playwright	All pages	Zero WCAG 2.1 AA violations on page render. Runs in CI on every PR.
Manual/Exploratory	QA engineer	Release candidates	Session-based testing per release cycle. Focus on edge cases and new features.

18. Deployment & CI/CD

Environments

Environment	URL / Host	Purpose	Deploy Trigger
Development	localhost:5173 (FE) / :8000 (BE)	Local dev with hot reload	Manual — developer runs locally
Staging	staging.inventorytracker.app	Pre-production QA, E2E tests, stakeholder review	Merge to develop branch (auto)
Production	app.inventorytracker.app	Live user traffic	Merge to main + manual approval gate in GitHub

CI/CD Pipeline (GitHub Actions)

Stage	Steps	On Failure
1. Lint	ESLint + Prettier (FE) · Ruff + MyPy (BE)	Block merge; fail fast
2. Test	Vitest (FE unit) · Pytest (BE unit + integration) · axe-core (a11y)	Block merge; publish test report
3. Build	Vite production build (FE) · Docker multi-stage image build (BE)	Block merge; notify Slack
4. Security Scan	Trivy container scan · npm audit · pip-audit	Block on High/Critical CVEs
5. Deploy Staging	Push Docker image · Deploy to Railway staging · Vercel preview URL	Notify Slack; skip E2E
6. E2E Tests	Playwright test suite against staging environment URL	Block production deploy; post report
7. Deploy Production	Manual approval gate → Railway prod + Vercel prod	Rollback to previous stable image
8. Post-Deploy	Smoke tests · Sentry release marker · Slack notification	Alert on-call engineer

Deployment Strategy Notes

- Blue-green: Railway maintains two production environments; load balancer flips in < 60s with no downtime
- DB migrations (Alembic) run before new app version receives traffic; always include reversible down() migration
- Feature flags via PostHog allow WIP features to be merged to main without user exposure
- Rollback procedure: repoint load balancer to previous environment; documented in runbook and tested monthly
- Health check: GET /health → { status: ok, db: ok, cache: ok } — verified by load balancer every 15s

19. Monitoring, Logging & Environment Variables

Monitoring & Alerting

Concern	Tool	Alert Threshold
Error tracking	Sentry	New error or rate spike > 5 errors/min → PagerDuty P1
API latency	Sentry Performance	p95 > 500ms sustained 2 min → Slack alert
Uptime	UptimeRobot + /health	Downtime > 1 min → PagerDuty P1
DB query perf	pg_stat_statements + Grafana	Query p95 > 100ms → review ticket created
Server resources	Railway metrics + Grafana Cloud	CPU > 80% for 5 min → scale-up alert
Auth security events	Supabase Auth logs + audit_log	> 10 failed logins/IP/min → auto-block + alert
Product analytics	PostHog	Feature adoption, funnel drop-offs — weekly review

Logging Strategy

- Application logs: structured JSON (Python logging + uvicorn); shipped to Grafana Loki
- Security logs: all auth events (login, logout, failed login, token refresh) logged with IP and user agent
- Audit logs: every CREATE/UPDATE/DELETE stored in audit_logs table — immutable, queryable by Admin
- Error logs: unhandled exceptions captured by Sentry with full stack trace and request context
- Performance logs: slow queries (> 50ms) logged via SQLAlchemy event listener; slow requests (> 500ms) by middleware
- Log retention: application logs 30 days; audit logs 12 months (compliance); security logs 90 days

Environment Variables

Variable	Dev	Staging	Prod	Purpose
DATABASE_URL	local postgres	Supabase staging	Supabase prod	PostgreSQL connection string
JWT_PRIVATE_KEY	dev key file	Secrets manager	Secrets manager	RS256 private key
JWT_PUBLIC_KEY	dev key file	Secrets manager	Secrets manager	RS256 public key
SUPABASE_URL	local / remote	Staging project URL	Prod project URL	Supabase project URL
SUPABASE_KEY	local anon key	Staging anon key	Prod service role key	Supabase API key
RESEND_API_KEY	test key	test key	live key	Email API key
REDIS_URL	redis://localhost	Upstash staging	Upstash prod	Cache connection
SENTRY_DSN	omit	staging DSN	prod DSN	Error tracking DSN
ALLOWED_ORIGINS	localhost:5173	staging domain	prod domain	CORS whitelist
SECRET_KEY	dev-secret	Secrets manager	Secrets manager	App signing secret
VITE_API_BASE_URL	localhost:8000	staging API URL	prod API URL	Frontend API base
VITE_POSTHOG_KEY	omit	staging key	prod key	Analytics key

20. Risks, Constraints & Timeline

Risk Register

ID	Risk	Like.	Impact	Mitigation
R-01	Concurrent stock updates causing inventory drift	Med	High	SELECT FOR UPDATE on inventory_levels rows; atomic UPDATE within transactions
R-02	Supabase Realtime latency causing missed low-stock alerts	Med	High	Secondary polling fallback every 60s; alert idempotency key prevents duplicates
R-03	PDF generation timeout for large reports	Low	Med	Offload to async background task; paginate source data; cache pre-built static reports
R-04	Cold start delay on free-tier hosting perceived as downtime	High	Med	Keep-alive ping every 5 min; UptimeRobot monitoring; upgrade to paid tier for SLA
R-05	Scope creep delaying v1.0 release	High	High	Strict change control: new features deferred to v2 backlog; weekly scope review
R-06	Developer leaves mid-project	Low	High	Documentation-first culture; pair programming for critical modules; mandatory code reviews
R-07	Supabase pricing/terms change	Low	Med	Service layer abstracts Supabase; migration to self-hosted PostgreSQL feasible

Constraints

Constraint	Detail
Budget	Launch on free tiers; upgrade budget ~\$50–100/month at 100+ active users
Timeline	v1.0 target: 12 weeks from project kickoff
Team Size	2 FE devs + 2 BE devs + 1 QA + 1 DevOps + 1 PM
Technology	Must use Supabase (infra mandate); Python 3.12+; React 18+; no proprietary SDKs
Compliance	PDPB (India) readiness; personal data stored in Supabase India region (ap-south-1)

Milestone Timeline

Week(s)	Milestone	Deliverables
1–2	Project Setup & Architecture	Repo, CI pipeline, DB schema v1, auth scaffolding, Docker setup
3–4	Core Backend APIs	Products, Categories, Inventory, Transactions CRUD + tests
5–6	Frontend Shell & Core Features	AppLayout, auth flow, product list/create, transaction form
7–8	Dashboard, Alerts & Export	KPI dashboard, low-stock alert engine, CSV/PDF export
9–10	Suppliers, POs & Notifications	Supplier module, purchase orders, notifications centre
11	QA, Load Testing & Security Scan	Full E2E suite, k6 load test pass, ZAP scan clean
12	Staging Hardening & Production Launch	UAT sign-off, production deploy, monitoring live, docs complete

21. Success Metrics & Acceptance Criteria

Success Metrics

Metric	Target	Measurement
Stockout incidents	80% reduction vs pre-launch baseline	Out-of-stock transaction events in DB
Manual reconciliation time	< 5 min/day per team	30-day post-launch user survey
System uptime	≥ 99.5% monthly	UptimeRobot monthly uptime report
API p95 read latency	< 200ms	Sentry Performance — monthly review
User adoption	90% of invited users active within 30 days	PostHog DAU/MAU ratio
SUS usability score	≥ 80 (Good)	10-question SUS survey at 30-day mark
P1 bug rate	< 2 per release cycle	GitHub Issues — bug severity labels
Test coverage	≥ 80% BE, ≥ 75% FE	Codecov report on each merge to main

Project-Level Definition of Done

- DOD-01:** All 18 Functional Requirements (FR-01 through FR-18) implemented and verified by QA
- DOD-02:** All Non-Functional Requirements met as measured in the pre-launch performance test
- DOD-03:** Test coverage thresholds met; no failing tests on main branch at time of release
- DOD-04:** Zero High or Critical findings in OWASP ZAP security scan on staging
- DOD-05:** WCAG 2.1 AA compliance verified via axe-core automated scan on all pages
- DOD-06:** API documentation auto-generated and live at /docs (Swagger UI) and /redoc
- DOD-07:** Monitoring live: Sentry, UptimeRobot, and Grafana dashboards all operational
- DOD-08:** Deployment runbook written and smoke-tested by a team member unfamiliar with the deploy process
- DOD-09:** Blue-green production switch tested; rollback procedure validated end-to-end
- DOD-10:** User manual and admin guide published in project wiki before launch

Maintenance & Disaster Recovery

Maintenance Plan

- Versioning: semantic versioning (MAJOR.MINOR.PATCH); CHANGELOG.md maintained per release
- P1 bugs (data loss, security breach): hotfix within 24 hours; deployed directly to production with approval
- P2 bugs (broken feature, incorrect data): fix within 5 business days in next sprint
- P3 bugs (cosmetic, minor): scheduled in next monthly minor release
- Dependency updates: Dependabot weekly PRs reviewed; security patches merged within 48h
- DB migrations: Alembic; always include reversible down() migration; tested on staging first
- On-call: 2 engineers on a 1-week rotation; PagerDuty escalation for P1 incidents
- Feature releases: monthly minor; quarterly major aligned to roadmap

Disaster Recovery

Scenario	Strategy	RTO	RPO
Application server failure	Blue-green switch / Railway auto-restart	< 60 seconds	No data loss
Database corruption	Supabase PITR (point-in-time recovery)	< 4 hours	< 24 hours
Full region failure	Restore from daily backup to alternate Supabase region	< 10 hours	< 24 hours
Accidental data deletion	Soft-delete by default; PITR up to 7 days back	< 2 hours	< 24 hours
CDN / Vercel outage	Fallback to self-hosted static files on Railway	< 30 minutes	No data loss
Compromised credentials	Rotate all secrets; revoke all refresh tokens; audit logs review	< 1 hour	No data loss

23. Future Enhancements & Appendix

v2.0 Roadmap

Priority	Feature	Description
High	AI Demand Forecasting	ML model predicts reorder qty based on sales velocity and seasonality
High	Mobile App (React Native)	Barcode scanning; offline-first transactions synced on reconnect
High	ERP / Accounting Integration	Two-way sync with Tally, QuickBooks, and Zoho Books
High	Multi-Currency Support	Product cost and PO value tracking with exchange rate API
Medium	Customer Order Portal	External customers view availability and raise orders
Medium	Advanced Analytics	Inventory turnover ratio, carrying cost, dead stock identification
Medium	Mandatory TOTP MFA	Google Authenticator for Admin + Manager roles
Medium	Bulk CSV Import	Initial product catalogue migration via file upload
Low	Outbound Webhooks	Configurable webhooks on low-stock, PO received, new transaction events
Low	White-labelling	Custom logo, colours, domain for enterprise customers

Technical Debt to Resolve Before v2.0

- Migrate async jobs from FastAPI BackgroundTasks to Celery + Redis worker queue
- Extract PDF report generation into a dedicated microservice to avoid API container memory pressure
- Replace offset pagination with cursor-based pagination for large dataset endpoints
- Add Supabase read replica routing for all reporting and export queries

Glossary

Term	Definition
SKU	Stock Keeping Unit — unique alphanumeric identifier for a product variant
RBAC	Role-Based Access Control — permissions assigned to roles, not individual users
RLS	Row-Level Security — PostgreSQL feature restricting row access per user/role at DB level
PITR	Point-in-Time Recovery — ability to restore a database to any point in the past
RTO	Recovery Time Objective — maximum acceptable downtime after a failure event
RPO	Recovery Point Objective — maximum acceptable data loss measured in time
LCP	Largest Contentful Paint — Core Web Vital measuring loading performance
TTFB	Time to First Byte — latency from request sent to first byte received from server
CLS	Cumulative Layout Shift — Core Web Vital measuring visual stability
INP	Interaction to Next Paint — Core Web Vital measuring responsiveness
SUS	System Usability Scale — standardised 10-item questionnaire; score 0–100
PO	Purchase Order — formal document issued to a supplier to procure goods
DAU/MAU	Daily / Monthly Active Users — engagement metric for product adoption

Term	Definition
JWT	JSON Web Token — compact, URL-safe token format used for authentication
DAST	Dynamic Application Security Testing — security scanning against a running application
PDPB	Personal Data Protection Bill — India's data privacy legislation
VU	Virtual User — simulated concurrent user in a load test (k6 terminology)
CSP	Content Security Policy — HTTP header controlling resource loading to prevent XSS
HSTS	HTTP Strict Transport Security — header forcing HTTPS for a specified duration

References

- FastAPI Documentation — <https://fastapi.tiangolo.com>
- Supabase Documentation — <https://supabase.com/docs>
- React 18 Documentation — <https://react.dev>
- TanStack Query Docs — <https://tanstack.com/query>
- OWASP Top 10 (2021) — <https://owasp.org/Top10>
- WCAG 2.1 Guidelines — <https://www.w3.org/TR/WCAG21>
- Core Web Vitals — <https://web.dev/vitals>
- RFC 7807 Problem Details — <https://tools.ietf.org/html/rfc7807>
- ReportLab Documentation — <https://www.reportlab.com/docs/reportlab-userguide.pdf>